



Sunbeam Institute of Information Technology
Pune and Karad

Algorithms and Data structures

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Collatz Conjecture

even — $\text{num} / 2$

odd — $\text{num} * 3 + 1$

8, 4, 2, 1

13, 40, 20, 10, 5, 16, 8, 4, 2, 1

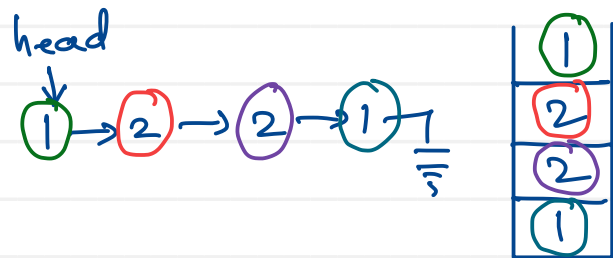
Palindrome Linked List

Given the head of a singly linked list, return true if it is a palindrome or false otherwise.

Example 1:

Input: head = [1,2,2,1]

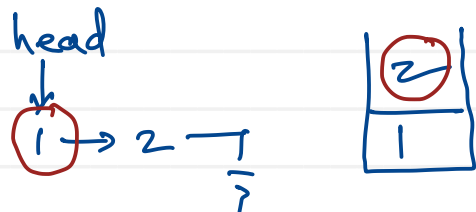
Output: true



Example 2:

Input: head = [1,2]

Output: false



to push : n

to pop : n

$$\underline{2n}$$

$$T \propto 2n$$

$$T(n) = O(n)$$

$$\text{Stack space} = n$$

$$\text{space} \propto n$$

$$S(n) = O(n)$$

```

boolean isPalindrome ( ) {
    Stack < Integer > st = new Stack < > ();
    Node trav = head;
    while (trav != null) {
        st.push ( trav.data );
        trav = trav.next;
    }
    trav = head
    while ( trav != null ) {
        if ( st.pop() != trav.data )
            return false;
        trav = trav.next
    }
    return true;
}
    
```

Middle of the Linked List

Given the head of a singly linked list, return the middle node of the linked list.

If there are two middle nodes, return the second middle node.

Example 1:

Input: head = [1,2,3,4,5]

Output: [3,4,5]

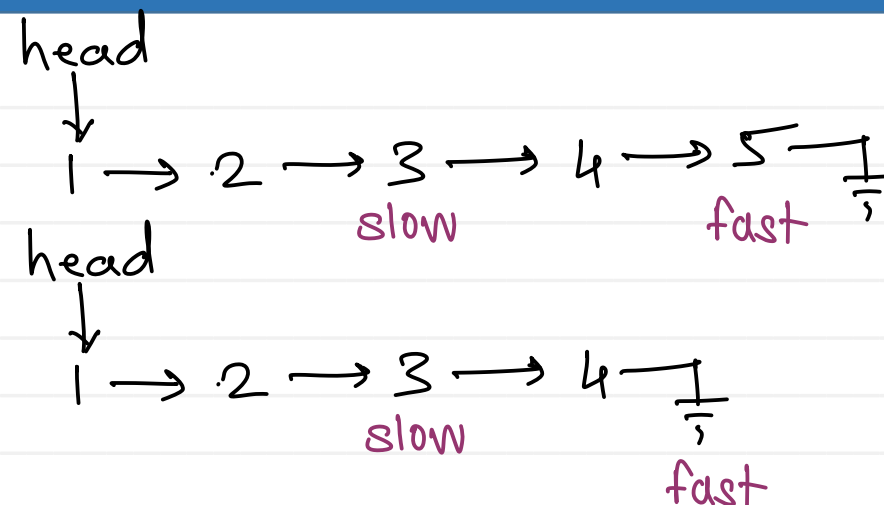
Explanation: The middle node of the list is node 3.

Example 2:

Input: head = [1,2,3,4,5,6]

Output: [4,5,6]

Explanation: Since the list has two middle nodes with values 3 and 4, we return the second one.



```
Node findMid( ) {  
    Node slow = head, fast = head;  
    while( fast != null && fast.next != null ) {  
        fast = fast.next.next;  
        slow = slow.next;  
    }  
    return slow;  
}
```

Linked List Cycle

Given head, the head of a linked list, determine if the linked list has a cycle in it.

Internally, pos is used to denote the index of the node that tail's next pointer is connected to. Note that pos is not passed as a parameter.

Return true if there is a cycle in the linked list.
Otherwise, return false.

Example 1:

Input: head = [3,2,0,-4], pos = 1

Output: true

Example 2:

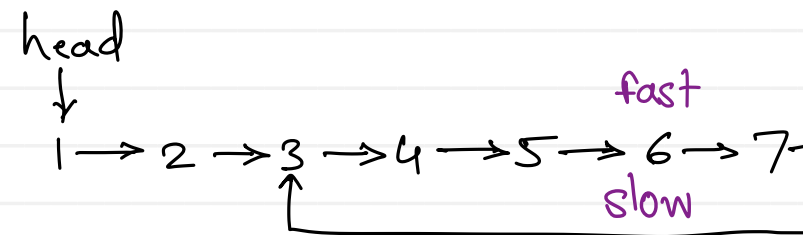
Input: head = [1,2], pos = 0

Output: true

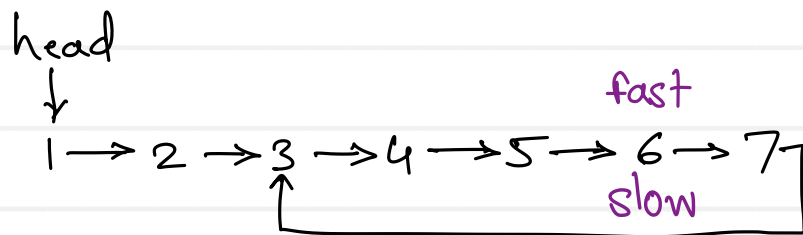
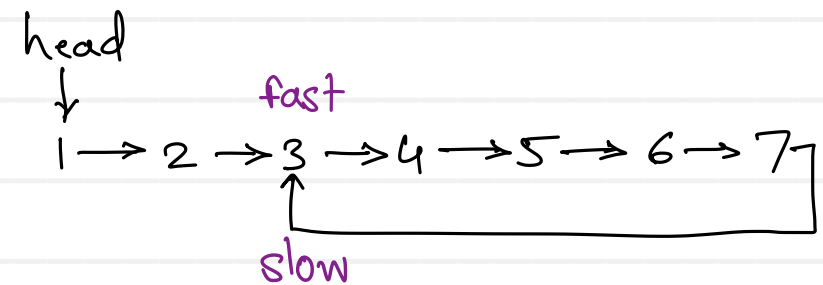
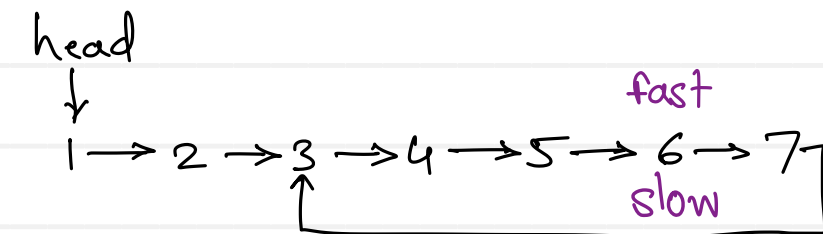
Example 3:

Input: head = [1], pos = -1

Output: false

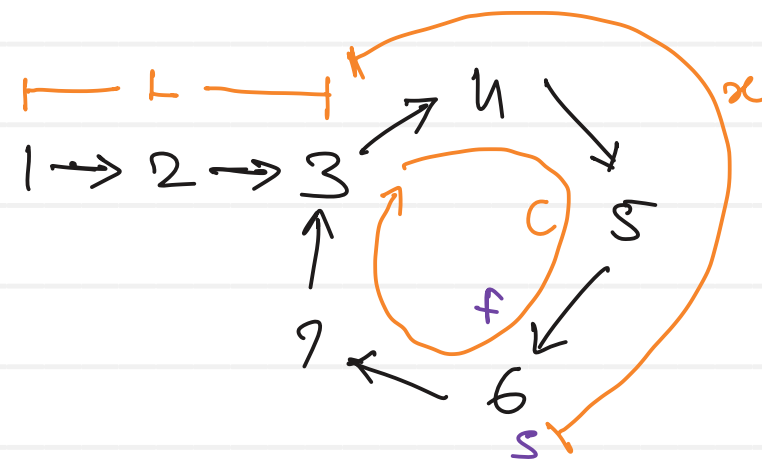


```
boolean hascycle(Node head) {  
    Node fast = head, slow = head;  
    while (fast != null && fast.next != null) {  
        fast = fast.next.next;  
        slow = slow.next;  
        if (fast == slow)  
            return true;  
    }  
    return false;  
}
```



S
count

Floyd cycle detection algorithm



$$\begin{aligned} \text{dist by slow} &= L + x \\ \text{dist by fast} &= L + x + nC \end{aligned}$$

no. of full traversal of the loop

$$\begin{aligned} \text{dist by fast} &= 2 * \text{dist by slow} \\ L + x + nC &= 2(L + x) \end{aligned}$$

$$nC = L + x$$

Two sum

Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Example 1:

Input: nums = [2,7,11,15], target = 9

Output: [0,1]

2, 7 ←

2, 11

2, 15

7, 11

7, 15

11, 15

Example 2:

Input: nums = [3,2,4], target = 6

Output: [1,2]

3, 2

3, 4

Example 3:

Input: nums = [3,3], target = 6

Output: [0,1]

3, 3 ←

Brute Force Approach

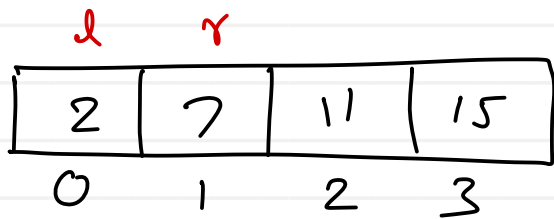
$$T(n) = O(n^2)$$

$$S(n) = O(1)$$

```
int[] twoSum(int[] nums, int target) {  
    for(i=0; i<nums.length-1; i++){  
        for(j=i+1; j<nums.length; j++){  
            if(nums[i]+nums[j]==target)  
                return new int[] {i, j};  
        }  
    }  
    return new int[] { };  
}
```

Two pointers Technique

- The two-pointer technique is a widely used approach to solving problems efficiently, which involves arrays or linked lists.
- This method involves traversing arrays or lists with two pointers moving at different speeds or in different directions.
- This technique is used to solve problems more efficiently than using a single pointer or nested loops.



target = 9

l	r	sum
0	3	17 >
0	2	13 >
0	1	9 =

- Find a pair of elements that sum up to a target.
 - Array: [2, 7, 11, 15]
 - Target Sum: 9
- Use two index variables **left** and **right** to traverse from both corners.
 - Initialize: $\text{left} = 0$, $\text{right} = n - 1$
 - Run a loop while **left** < **right**, do the following inside the loop
- Compute current sum, **sum** = arr[left] + arr[right]
- If the **sum** equals the **target**, we've found the pair.
- If the **sum** is less than the **target**, move the **left** pointer to the right to increase the **sum**.
- If the **sum** is greater than the **target**, move the **right** pointer to the left to decrease the **sum**.

Two sum

sorted

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9`

Output: `[0,1]`

Example 2:

Input: `nums = [1,2,4]`, `target = 6`

Output: `[1,2]`

Example 3:

Input: `nums = [3,3]`, `target = 6`

Output: `[0,1]`

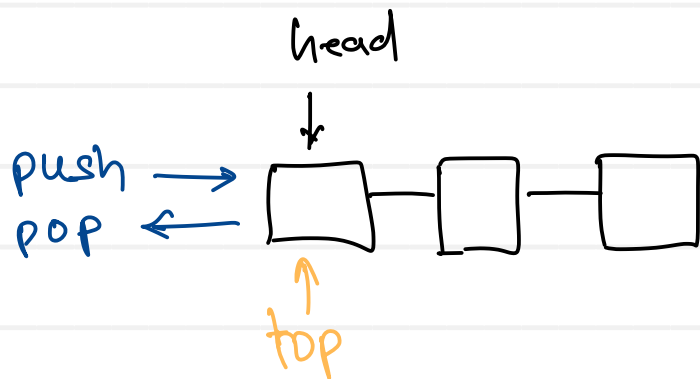
```
int[] twoSum(int[] nums, int target) {  
    int left = 0, right = nums.length - 1;  
    while (left < right) {  
        sum = nums[left] + nums[right];  
        if (sum == target)  
            return new int[] {left, right};  
        else if (sum < target)  
            left++;  
        else  
            right++;  
    }  
    return new int[] { };  
}
```

Stack

- single end : top
- LIFO

1. Add/Delete first
2. Add/Delete last

Time complexity of operations = $O(1)$

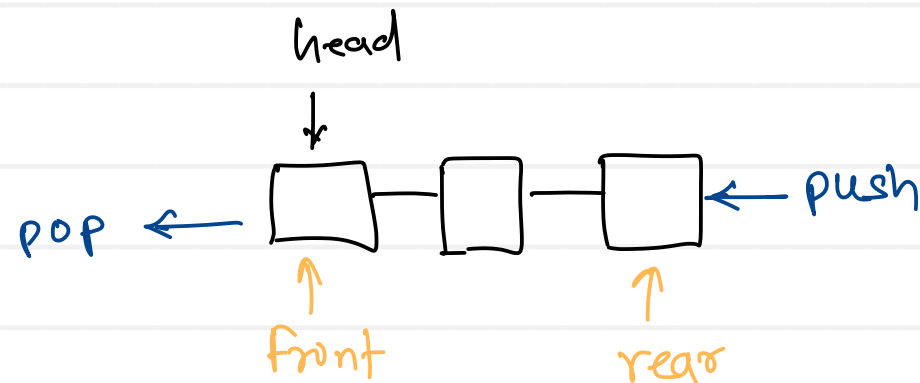


Queue

- two ends : front, rear
- FIFO

1. Add first, Delete last
2. Add last, Delete first

Time complexity of operations = $O(1)$



- Stack is a linear data structure which has only one end - top
- Data is inserted and removed from top end only.
- Stack works on principle of “Last In First Out” / “LIFO”

Capacity = 4

Operations :

1. Push -

- reposition top (inc)
- add value at top index

2. Pop -

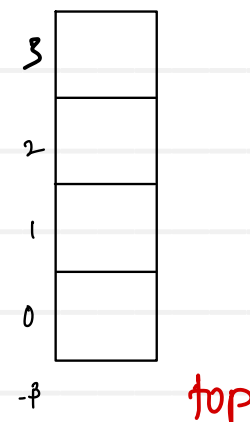
- reposition top (dec)

3. Peek -

- read/return data of top index

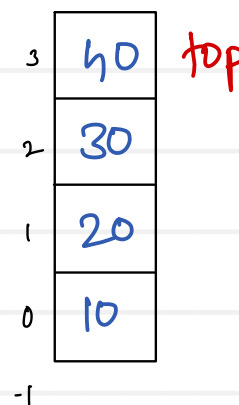


Empty



top == -1

Full



top == size - 1

- all operations of stack are performed in $O(1)$ time complexity.

Ascending stack

$top = -1$

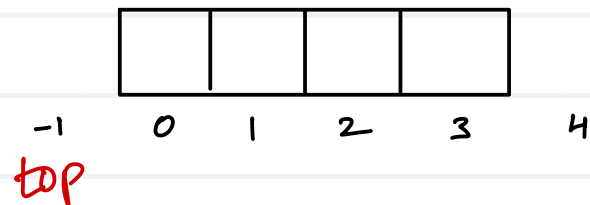
push : $top++$
 $arr[top] = value$

pop : $top--$

peek : $arr[top]$

Empty : $top == -1$

Full : $top == size-1$



Descending Stack

$top = size$

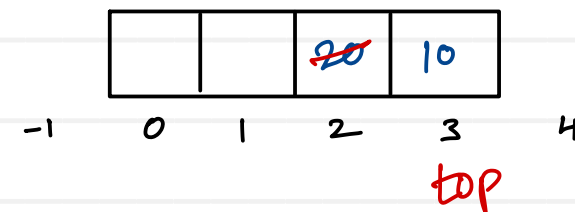
push : $top--$
 $arr[top] = value$

pop : $top++$

peek : $arr[top]$

Empty : $top == size$

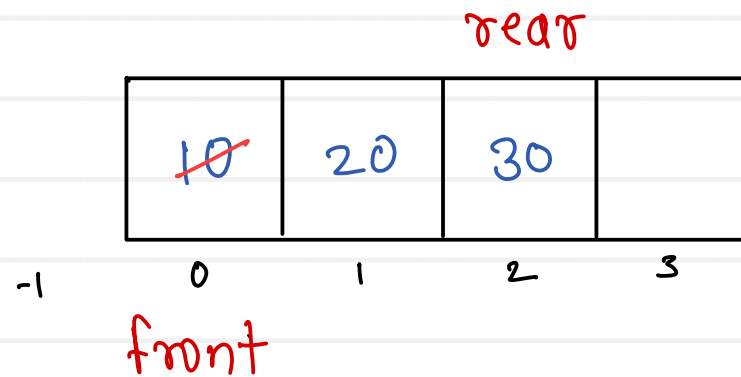
Full : $top == 0$



Linear queue

- linear data structure which has two ends - front and rear
- Data is inserted from rear end and removed from front end
- Queue works on the principle of “First In First Out” / “FIFO”

Capacity = 4



Operations :

1. Push/enqueue / offer :

- a. reposition rear (inc)
- b. add value at rear index

2. Pop/dequeue / poll :

- a. reposition front (inc)

3. Peek :

- a. read/return value of front end
(front + 1 index)

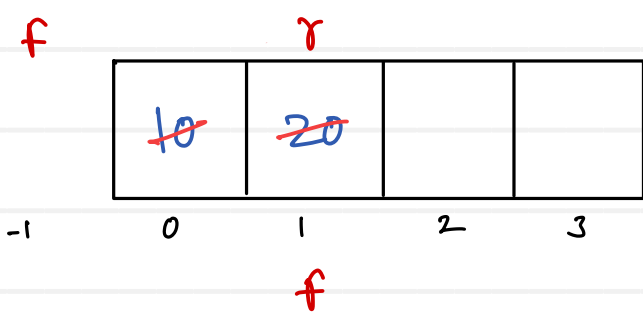
- all operations of queue are performed in $O(1)$ time complexity.

Linear queue - Conditions

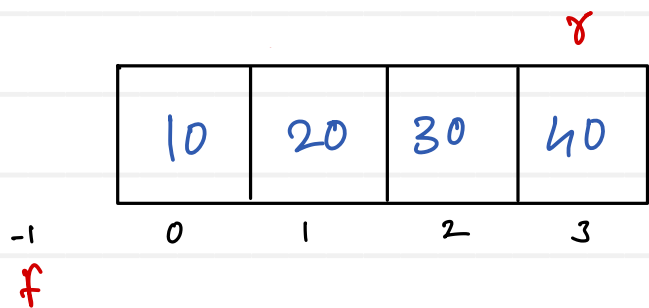
Empty



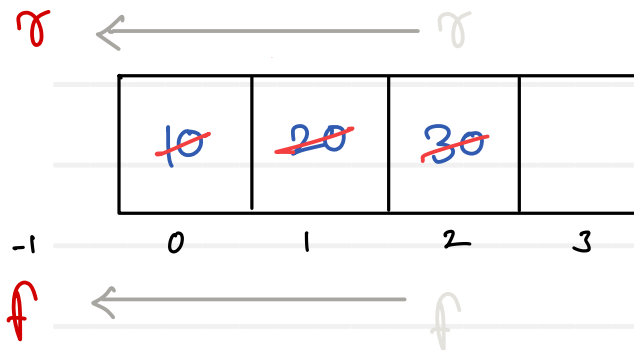
$front == rear$



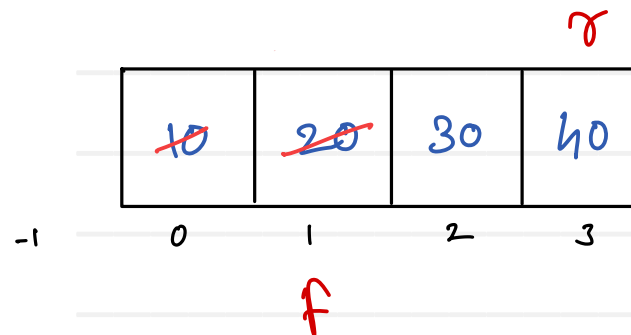
Full



$rear == size - 1$



if (front == rear)
front = rear = -1



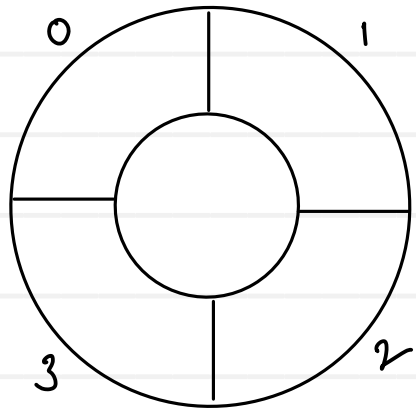
- if linear queue is partially empty (few initial locations are empty), we can not reuse those empty locations until queue will be completely empty

solution is circular queue

← this leads to poor memory utilization.

Circular queue

Capacity = 4



$$\text{front} = (\text{front} + 1) \% \text{size}$$

$$\text{rear} = (\text{rear} + 1) \% \text{size}$$

$$\text{front} = \text{rear} = -1$$

$$= (-1 + 1) \% 4 = 0$$

$$= (0 + 1) \% 4 = 1$$

$$= (1 + 1) \% 4 = 2$$

$$= (2 + 1) \% 4 = 3$$

$$= (3 + 1) \% 4 = 0$$

operation:

1. Enqueue

- reposition rear (inc)
- add data at rear index

2. Dequeue

- reposition front (inc)

3. Peek

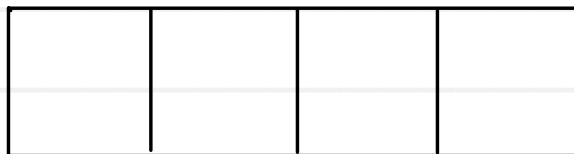
- read/return data of front end
(front+1) index

- all operations of queue are performed in $O(1)$ time complexity

Circular queue - Conditions

Empty

rear



-1

0

1

2

3

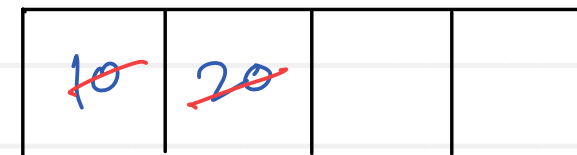
front

$front == rear \ \&\& \ rear == -1$

rear

←

rear



-1

0

1

2

3

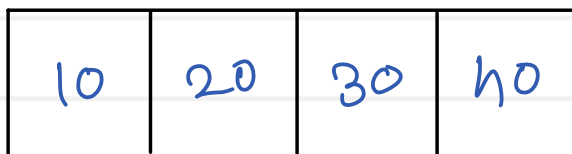
front

←

front

Full

rear



-1

0

1

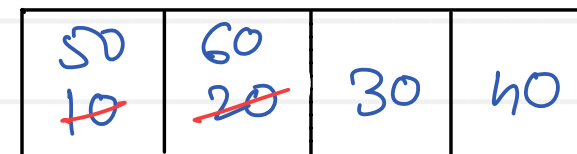
2

3

front

$(front == -1 \ \&\& \ rear == size - 1) \ || \ (front == rear \ \&\& \ rear != -1)$

rear



-1

0

1

2

3

front

Applications of Queue

- job queue, ready queue or waiting queue in OS
- jobs submitted to printer [spooler directory]
- call to customer care
- token systems
- in networks to modify common files
- advanced data structures for traversal
(BFS - queue)

Stack Applications

- Lexical analysis in compilers
 - parenthesis balancing
- function activation records
 - ↳ control the flow program
- traversal in advanced data structure
 - DFS - stack
 - preorder/inorder/postorder - stack
- Expression conversion & evaluation

Stack

- Parenthesis balancing
- Expression conversion and evaluation
- Function calls
- Used in advanced data structures for traversing
- **Expression conversion and evaluation:**
 - Infix to postfix
 - Infix to prefix
 - Postfix evaluation
 - Prefix evaluation

Queue

- Jobs submitted to printer
- In Network setups – file access of file server machine is given to First come First serve basis
- Calls are placed on a queue when all operators are busy
- Used in advanced data structures to give efficiency.
- Process waiting queues in OS

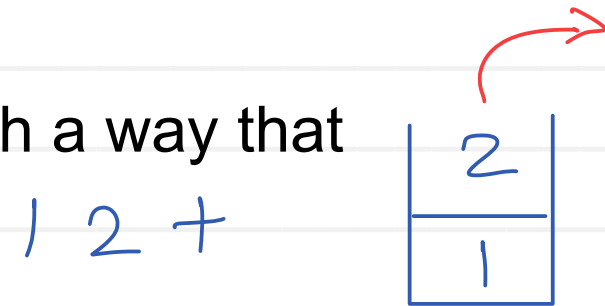
Expression : combination of operands & operators

Types :

1. Infix : $a + b$ $\rightarrow$ human
 2. Prefix : $+ a b$
 3. Postfix : $a b +$
- } $\rightarrow$ machine $\rightarrow$ CPU $\rightarrow$ ALU

Postfix Evaluation

- Process each element of postfix expression from left to right
- If element is operand
 - Push it on a stack
- If element is operator
 - Pop two elements (Operands) from stack, in such a way that
 - Op2 – first popped element
 - Op1 – second popped element
 - Perform current element (Operator) operation between Op1 and Op2
 - Again push back result onto the stack
- When single value will remain on stack, it is final result
- e.g. 4 5 6 * 3 / + 9 + 7 -

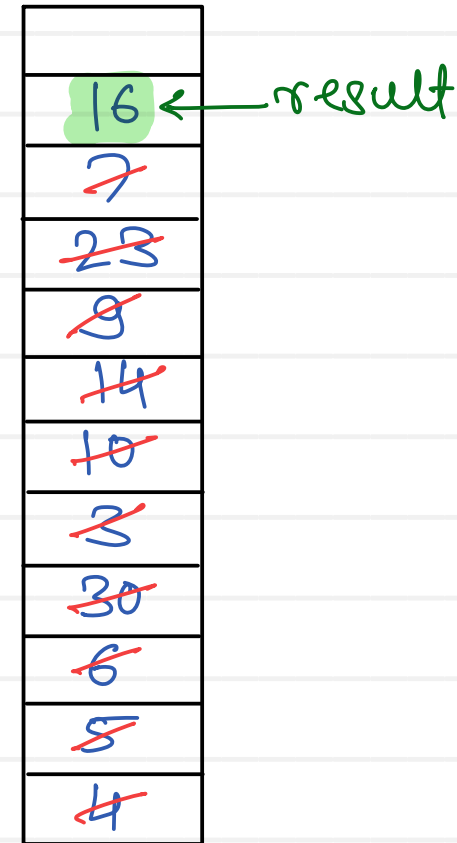


Postfix evaluation

Postfix expression : 4 5 6 * 3 / + 9 + 7 -

left $\longrightarrow$ right

- ⑤ $23 - 7 = 16$
- ④ $14 + 9 = 23$
- ③ $4 + 10 = 14$
- ② $30 / 3 = 10$
- ① $5 * 6 = 30$



stack

Prefix Evaluation

- Process each element of prefix expression from right to left
- If element is operand
 - Push it on a stack
- If element is operator
 - Pop two elements (Operands) from stack, in such a way that
 - Op1 – first popped element
 - Op2 – second popped element
 - Perform current element (Operator) operation between Op1 and Op2
 - Again push back result onto the stack
- When single value will remain on stack, it is final result
- e.g. - + + 4 / * 5 6 3 9 7



Prefix expression : - + + 4 / * 5 6 3 9 7

left ← right

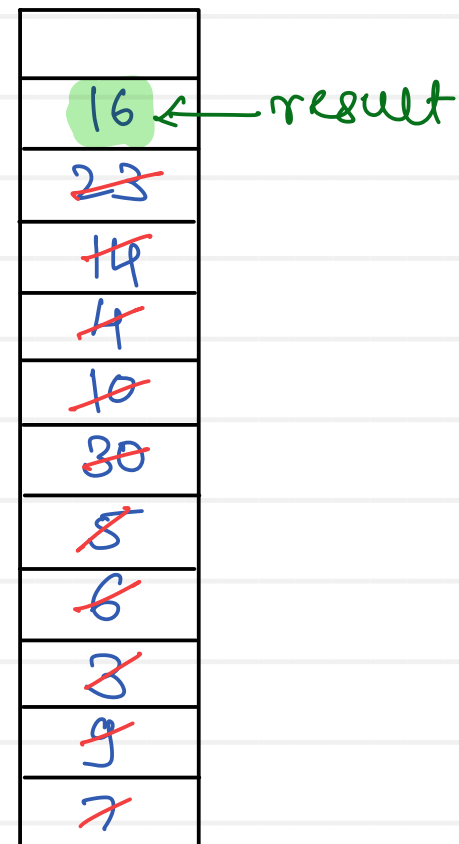
⑤ $23 - 7 = 16$

④ $14 + 9 = 23$

③ $4 + 10 = 14$

② $30 / 3 = 10$

① $5 * 6 = 30$



stack

Infix to Postfix Conversion

- Process each element of infix expression from left to right
- If element is Operand
 - Append it to the postfix expression
- If element is Operator
 - If priority of topmost element (Operator) of stack is greater or equal to current element (Operator), pop topmost element from stack and append it to postfix expression
 - Repeat above step if required
 - Push element on stack
- Pop all remaining elements (Operators) from stack one by one and append them into the postfix expression
- e.g. $a * b / c * d + e - f * h + i$

Infix to Postfix conversion

Infix expression : $1 \$ 9 + 3 * 4 - (6 + 8 / 2) + 7$

left $\longrightarrow$ right

Postfix expression : $19\$34*+682/+ -7+$

$+$
$+$
$+$
$($
$/$
$*$
$+$
$$$

Infix to Prefix Conversion

- Process each element of infix expression from right to left
- If element is Operand
 - Append it to the prefix expression
- If element is Operator
 - If priority of topmost element of stack is greater than current element (Operator), pop topmost element from stack and append it to prefix expression
 - Repeat above step if required
 - Push element on stack
- Pop all remaining elements (Operators) from stack one by one and append them into the prefix expression
- Reverse prefix expression
- e.g. $a * b / c * d + e - f * h + i$

Infix to Prefix conversion

Infix expression : $1 \$ 9 + 3 * 4 - (6 + 8 / 2) + 7$

left $\leftarrow$ right

Expression : $7 2 8 / 6 + 4 3 * 9 1 \$ + - +$

Prefix expression : $+ - + \$ 1 9 * 3 4 + 6 / 8 2 7$

\$
+
*
-
+
/
)
+

(

Prefix to Postfix

- Process each element of prefix expression from right to left
- If element is an Operand
 - Push it on to the stack
- If element is an Operator
 - Pop two elements (Operands) from stack, in such a way that
 - Op1 – first popped element
 - Op2 – second popped element
 - Form a string by concatenating Op1, Op2 and Opr (element)
 - String = “Op1+Op2+Opr”, push back on to the stack
- Repeat above two steps until end of prefix expression.
- Last remaining on the stack is postfix expression
- e.g. $* + a b - c d$

Postfix to Infix

- Process each element of postfix expression from left to right
- If element is an Operand
 - Push it on to the stack
- If element is an Operator
 - Pop two elements (Operands) from stack, in such a way that
 - Op2 – first popped element
 - Op1 – second popped element
 - Form a string by concatenating Op1, Opr (element) and Op2
 - String = “Op1+Opr+Op2”, push back on to the stack
- Repeat above two steps until end of postfix expression.
- Last remaining on the stack is infix expression
- E.g. a b c - + d e - f g - h + / *

Valid Parantheses

Given a string *s* containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

- Open brackets must be closed by the same type of brackets.
- Open brackets must be closed in the correct order.
- Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: *s* = "()"

Output: true

Example 2:

Input: *s* = "()[]{}"

Output: true

Example 3:

Input: *s* = "([]"

Output: false

Example 4:

Input: *s* = "([)]"

Output: true

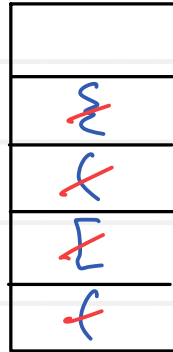
1. create stack to push brackets
2. traverse string from left to right
 - 2.1 if bracket is opening
then push it on stack
 - 2.2 if bracket is closing
 - 2.2.1 if stack is empty, return false.
 - 2.2.2 if stack is not empty,
 - pop one bracket from stack,
 - if they are matching, continue
 - if they are not matching, return false.
3. if stack is not empty, return false
4. if stack is empty, return true

Parenthesis balancing using stack (lexical analysis)

([] ({ }))



] == [
 } == {
) == (
) == (



stack

([] ({ }])



] == [
 } == {
] != (



stack

opening

(	[	{
0	1	2

closing

)	]	}
0	1	2

string

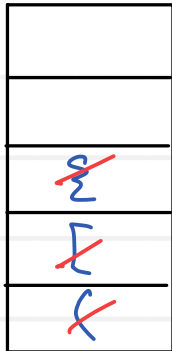
↓
indexOf()

↓
returns index of char
returns -1 if char
not found

([] { })



] == [
 } == {
) == (
) ?

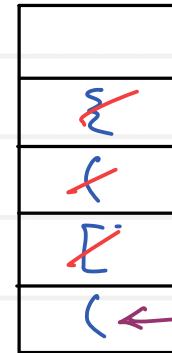


stack

([] ({ })



] == [
 } == {
) == (



stack